Sara: A Numba-CUDA Ray Tracer

Matěj Zeman*
Supervisor: Ing. Radek Richtr, Ph.D.
Course: NI-PG1: Počítačová Grafika 1

May 2026

Abstract

This report presents a simple ray tracer implemented in Python, utilizing Numba CUDA for GPU acceleration. The renderer supports most of the ray-tracing pipeline including diffuse textures, reflective and transmissive materials, area light integration, and ACES-based colour management with Intel Open Image Denoise [5] pass. A key contribution is the development and comparison of three BVH construction strategies: naive triangle enumeration, median splitting, and Surface Area Heuristic (SAH) with binning. The project also documents practical limitations of using a Python-to-CUDA JIT compiler.

1 Engine Structure & Implementation Challenges

1.1 Core Architecture

Sara is built around Numba's CUDA JIT compiler, which translates a subset of Python and NumPy into PTX. The rendering pipeline runs primarily on the GPU, with the CPU serving optional support tasks:

- **GPU**: Ray tracing (primary, secondary, and shadow rays), BVH traversal via per-thread stack, Möller–Trumbore intersection [2], material shading, HDR framebuffer, tonemapping, OIDN denoising.
- **CPU**: Scene loading via `tinyobjloader` (C++ with pybind11) [7], BVH construction, optional post-processing (tonemapping, OIDN denoise, sRGB conversion).

Numba vs CUDA C++: This project implements a ray tracer through Python's JIT layer rather than native CUDA C++ or OptiX. Numba lacks the mature ecosystem of C++ ray tracers: no GPU-side sorting (forcing SAH BVH on CPU), no acceleration libraries, and no native GPU debugger. Type inference silently promotes `float32` to `float64` through Python's scalar rules. Nsight Compute [4] and printed values are the only realistic debugging steps.

Scene data is stored in contiguous 1D NumPy arrays accessed via named field constants. BVH nodes and materials use a struct-of-arrays layout where each column represents one property across all entities. Triangle geometry and per-pixel data (framebuffer, textures) group channels together since they are always accessed as complete vectors. This hybrid layout eliminates Python object overhead while keeping channel-wise data co-located where needed.

The kernel uses a $16 \times 16 = 256$-thread block, with each thread rendering one pixel. Samples accumulate iteratively with HDR framebuffer averaged at the end. An ACEScg-to-sRGB colour pipeline handles gamut mapping with several tonemapper options (Section 5.1).

1.2 Implementation Challenges

- **No GPU sorting:** Numba lacks GPU sort primitives, forcing SAH BVH construction onto the CPU (e.g., 6.69 s for dragon).
- **Iterative traversal:** BVH traversal uses a per-thread local stack (`STACK_SIZE` = 64) to avoid recursion depth limits.
- **Register pressure & occupancy:** The mega kernel carries many variables per thread. On Ampere (RTX 3090), high register pressure reduces occupancy versus Ada Lovelace (RTX 40-series) [3]. Larger blocks exceed register limits and fail to launch. The 8×8 configuration yields better throughput on the RTX 4070 Ti (11% faster), confirming 16×16 was chosen for higher occupancy at the cost of raw speed.
- **Float64 propagation:** The Python `float` type promotes NumPy expressions to `float64`, causing slow double-precision math on the GPU. This is mitigated using `float32()` casts and typed aliases in `constants.py`.
- **Precision and epsilons:** Many epsilon values manage float32 numerical limits: determinant culling (10^{-9}), surface offset and near-zero t filtering (10^{-5}), barycentric weight validation (10^{-7}), smaller division-by-zero guards (10^{-10}), a shadow ray distance capping (10^{-4}), and path throughput threshold (10^{-2}). The shadow origin offset uses Chiang's analytical terminator [6] to project the hit point onto face planes, weighted by barycentric coordinates and clamped to 1 cm. These epsilons solve all potential numerical issues: peter panning at shallow ray angles, self-shadowing from accumulated precision loss, and grazing (almost orthogonal) angle valid hits rejected by the t-min filter.
- **Numba classes:** Numba's `types.ClassType` support on the GPU is experimental and can cause performance regressions through hidden memory allocations and type dispatch overhead. The project avoids classes entirely, keeping the code close to C-style procedural code with flat arrays and simple function dispatch.

*Student, CTU FIT Prague, `zemanm40@fit.cvut.cz`

1.3 Divergence & Wavefront Architecture

Baseline profiling reveals significant warp divergence: 8.38 active threads per warp (26.2% utilization) on the dragon scene. Shadow rays are coherent, reflective/transmissive rays diverge, and BVH traversal is incoherent. An alternative *wavefront* architecture on the `gpu` branch used a two-pass approach that failed on Ampere due to register spilling (4.68 threads/warp).

2 Advanced Methods

2.1 Sampling & Denoising

Stochastic jittered sampling samples each pixel `SAMPLES` times with uniform jitter. **Intel OIDN denoising** [5] uses the `RT` filter with a `color` buffer in `hdr` mode. At 16 SPP, OIDN produces practical results. The ACEScg colour pipeline operates in linear space (Section 5.1). A 3D LUT offloads colour conversions from the per-pixel kernel.

2.2 Compilation Model & Memory

Numba uses Python decorators (`@cuda.jit`, `fastmath=True`) evaluated at JIT. The compiler infers types from NumPy dtypes, lowers to LLVM IR, then generates PTX. Dead code elimination prunes `if FLAG:` blocks at compile time. NumPy's promotion rules can elevate `float32` to `float64`. All literals are wrapped in `float32()`. Stochastic sampling uses per-thread seeds computed from pixel coordinates and sample index. The `cuda.shared.array` API allocates block-level shared memory, while `cuda.local.array` provides per-thread private storage. Exceeding register limits causes spills to global memory. `STACK_SIZE` is kept at 64 to prevent register exhaustion during BVH traversal.

3 Acceleration Data Structure

3.1 Methodology

We implemented three top-down BVH split strategies [1]. GPU-side BVH traversal is iterative, using a per-thread local stack in depth-first order.

3.2 Construction Strategies

Three strategies were implemented on the CPU (with `@njit` decorator):

1. **Naive**: All triangles in a single leaf node. Tests every triangle per ray. Impractical beyond a handful of primitives.

2. **Spatial Median**: Splits at the geometric midpoint along the longest axis. Produces a balanced tree quickly but ignores traversal cost asymmetry.

3. **SAH + Binning**: Minimises the cost function

$$C(s) = 1.0 + \frac{N_{\text{left}} \cdot A_{\text{left}} + N_{\text{right}} \cdot A_{\text{right}}}{A_{\text{parent}}}$$

   where A is child AABB surface area and N is the number of triangles in each child. The constant 1.0 represents the traversal cost of the current node. Binning (8 bins per axis) avoids evaluating every split point [13], trading minor quality loss for significantly faster construction.

Exhaustive SAH evaluates every triangle as a split candidate ($O(N^2)$ per node). Binning maps triangles into $B = 8$ bins per axis with prefix/suffix accumulations, evaluating all $B - 1$ inter-bin splits in $O(B)$ time. This drops construction complexity from $O(N^2)$ to $O(N \cdot \text{depth})$. Table 4.1 shows exhaustive SAH improves only 13% over binned SAH (775 vs. 892 mean node visits). Dragon was not benchmarked with median-split or exhaustive SAH due to build time constraints.

The BVH builder runs single-threaded on the CPU as Numba parallel JIT does not support dynamic task spawning. After construction, the tree is *flattened* into `bvh_nodes` (8 floats per node) and `tri_ids` arrays for O(1) child access on the GPU.

4 Performance Results

Scene	Config	Build (s)	Node	Tri	Shadow	Traverse
bunny	sah-binning	4.16	892.42	77.26	19.82	29.78
bunny	median-split	5.31	2903.43	413.77	19.83	97.69
bunny	sah-exhaustive	508.92	775.11	79.97	19.82	25.70
box-spheres	sah-binning	3.39	454.18	52.85	13.34	15.26
box-spheres	median-split	3.38	1244.35	251.61	13.35	44.90
box-spheres	sah-exhaustive	3.84	342.40	51.83	13.35	11.73
dragon	sah-binning	6.69	52.39	7.97	10.68	10.53

Table 4.1: Per-pixel mean metrics (1024^2, 16 SPP).

SAH + binning yields the lowest triangle test counts. On bunny, SAH reduces tests from 413.77 (median) to 77.26 (**5.4**×). Exhaustive SAH takes 508 s vs. 4 s for binned, confirming the 8-bin approximation's efficiency. On dragon (871k triangles), exhaustive SAH would take 90 min while binned SAH completes in 6.69 s, avoiding the impracticality of naive rendering.

4.1 BVH Construction Metrics

Table 4.2 details the BVH trees after SAH construction. Dragon requires 6.69 s and produces 932,985 nodes for 871,322 triangles (depth 32, 1.9 prims/leaf). Build time is high due to single-threaded Python construction.

Scene	Nodes	Internal	Leaves	Depth	Prims/leaf
bunny	76,907	38,453	38,454	8.3	1.8
box-spheres	2,183	1,091	1,092	5.7	2.0
dragon	932,985	466,492	466,493	32	1.9

Table 4.2: BVH construction statistics (SAH + binning).

4.2 Render Time Logs

Sara implements BVH traversal entirely in software via Numba-CUDA, without OptiX.

Table 4.3 breaks down timings on an NVIDIA RTX 3090. GPU tonemapping executes in a single fused kernel combining exposure, the ACEScg LUT, and gamma correction. Without BVH the bunny scene requires 36 s, versus 0.11 s with SAH BVH and denoising (BVH build time of 7.34 s excluded). Shadow rays represent 37–42% of all rays.

Stage	Time (s)
BVH build (bunny)	7.34
Render (bunny, BVH, 16 SPP)	0.09
Tonemap + postprocess (GPU)	0.01
Denoise (GPU, OIDN RT)	0.02
Total render	**0.11**

Table 4.3: Bunny scene render timing breakdown (RTX 3090, 1024^2, 16 SPP, denoise ON). BVH build time listed separately.

5 Visual Outputs

All renders use stochastic jittered sampling and OIDN denoising (16 SPP).

Figure 5.1: Scene renders (16 SPP, 1024^2, denoise ON).

5.1 Tonemapper Comparison

The renderer provides six ways of tonemapping: *custom* (ACEScg → sRGB 3D LUT with Oklab hue compression [11], GPU), *Khronos PBR neutral* (per-channel glTF neutral), *Narkowicz* (per-channel ACES fit [8]), *Hill* (analytical ACES approximation [9]), *none* (hard clip), and *magenta* (debug overshoot).

Custom (Oklab 3D LUT [11]). The custom tonemapper bakes an entire ACEScg → sRGB pipeline into a 32^3 3D LUT spanning -10 to +10 EV. The offline pipeline applies:

- **CAT02 chromatic adaptation** [14] to convert ACEScg white to linear sRGB.
- **Asymmetrical crosstalk matrix** simulating physical film emulsion. Red bleeds into green, green into blue, and blue into red, as in modern filmic transforms [15].
- **Binary-search gamut compression** in Oklab [11]: the algorithm bisects the chroma scale (a, b channels) for 15 iterations until no sRGB channel falls below zero.
- **White highlight desaturation** using a hermite smoothstep that gradually reduces chroma as lightness rises from 1.5 to 2.0, mimicking photographic highlight roll-off [15].
- **Uchimura piecewise tonemapping** [12] applied to the per-pixel maximum channel value (achromatic norm) with an exposure pre-scale of 0.6 (equivalent to the Academy RRT). The ratio of mapped-to-original norm is applied uniformly across all channels, preserving relative RGB proportions.
- **sRGB gamma encode** via the standard 2.2 power curve [16].

The resulting LUT is loaded at runtime and sampled with trilinear interpolation on the GPU kernel.

Khronos PBR neutral. Per-channel glTF neutral curve: clamps negatives, applies a rational function for highlight compression. Each channel is processed independently: the toe preserves proportional contrast near zero, the shoulder compresses above unity. The rational function

$$f(x) = \frac{x \cdot (0.447x + 0.465)}{x \cdot (0.447x - 0.314) + 1.0}$$

with a numerical guard $0.465x$ below 10^{-4} provides C^1 continuity and approximates the human luminance response. The curve anchors at exposure 1/3, mapping middle-grey to 0.5. Uniform per-channel scaling maintains the white point, though it lacks gamut mapping and can clip saturated highlights.

Narkowicz / Hill (ACES [10]). Both apply the analytical ACES 1.0 fit per channel, bypassing the sRRT and ODT stages. Narkowicz [8] uses

$$f(x) = \frac{0.536x + 0.016}{0.467x + 0.983}$$

for punchier contrast; Hill [9] adjusts the denominator to $D = 1.015$ for smoother highlight roll-off. Both require only multiply-add operations per channel, making them extremely efficient. Neither performs gamut compression or chromatic adaptation, so out-of-gamut colours compress along straight lines through the origin, causing mild desaturation of saturated reds and cyans. In practice this manifests as a faint purple fringe on specular highlights, visible in the comparison figure.

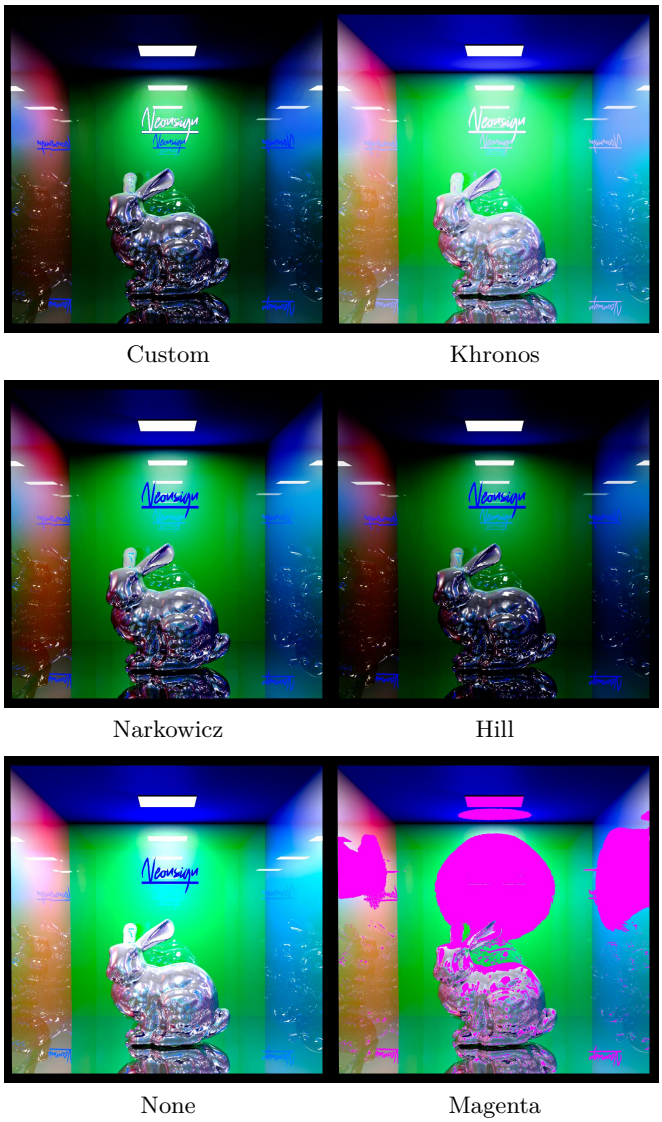

Figure 5.2: Tonemapper comparison (Enhanced bunny scene).

6 Conclusion

On the bunny scene (RTX 3090, BVH), shadow rays represent 37.2% of workload with 14.7 node visits per hit vs. 279,967 without BVH.

Sara demonstrates that Numba-CUDA achieves high-quality results with SAH + binning BVH (5× fewer triangle tests vs. median) and ACES colour pipeline. Future work: GPU-side BVH construction, volume rendering, displacement-mapped geometry.

Failure cases. Deep transmissive scenes can saturate the bounce limit. BVH stacks exceeding `STACK_SIZE` = 64 silently truncate paths. OIDN smears fine detail at low SPP without normal/albedo buffers. Float32 precision causes self-shadowing artifacts.

References

[1] Matt Pharr, Wenzel Jakob, and Greg Humphreys. *Physically Based Rendering: From Theory to Implementation*. Morgan Kaufmann, 3rd edition, 2016.

[2] Tomas Möller and Ben Trumbore. Fast, Minimum Storage Ray-Triangle Intersection. *Journal of Graphics Tools*, 2(1):21–28, 1997.

[3] NVIDIA Corporation. *CUDA C++ Programming Guide v12*. CUDA C++ Programming Guide, 2023.

[4] NVIDIA Corporation. *Nsight Compute—GPU Profiler for CUDA Applications*. NVIDIA Documentation, 2024.

[5] Intel Corporation. Intel Open Image Denoise: High-Performance Denoising Library for Ray Tracing. openimagedenoise.org, 2023.

[6] M. J.-Y. Chiang, Y. K. Li, B. Burley. Taming the Shadow Terminator. *ACM SIGGRAPH 2019 Talks*, Article 71, July 2019. doi:10.1145/3306307.3328172.

[7] Syoyo Fujita. TinyObjLoader: Tiny but Powerful Wavefront OBJ & MTL Loader. GitHub repository, 2023.

[8] Krzysztof Narkowicz. ACES Filmic Tone Mapping Curve. Knarkowicz Blog, 2016.

[9] Stephen Hill. ACES Filmic Tone Mapping Implementation in HLSL. BakingLab — ACES Tonemapping HLSL, 2016.

[10] The Academy of Motion Picture Arts and Sciences (AMPAS). SMPTE ST 2065-1: Academy Color Encoding Specification (ACES). ACES Central — SMPTE ST 2065-1, 2021.

[11] Björn Ottosson. Oklab: A perceptually uniform color space. Project page, 2020.

[12] Hajime Uchimura.
HDR theory and practice: Uchimura tone mapping curve. CEDEC 2017, Tokyo. Reference GLSL implementation.

[13] Ingo Wald. On fast construction of SAH-based bounding volume hierarchies. *IEEE Symp. Interactive Ray Tracing*, 2007. doi:10.1109/RT.2007.4342591.

[14] Nathan Moroney, Mark D. Fairchild, Robert W. G. Hunt, Changjun Li, M. Ronnier Luo, and Todd Newman. The CIECAM02 Color Appearance Model. *IS&T/SID 10th Color Imaging Conference*, 2002.

[15] Jed Smith. AgX – A minimal display transform for digital content creation. GitHub repository, 2023.

[16] IEC. IEC 61966-2-1:1999. sRGB colour space. IEC Webstore, 1999.